ShadowMem — Hackathon Defense & UI Guide

Plain-Language Screen-by-Screen Walkthrough, Formulas, and Live Evaluator QA

Who this guide is for: Presenters defending ShadowMem to hackathon judges. Every UI element, button, metric, badge, and modal is explained in plain English with zero unexplained jargon. Every item is tagged with its real operating status: **REAL**, **DERIVED**, **DECORATIVE**, **NOT WIRED**, or **HARDCODED**.

0. The Big Picture in 10 Lines

1. **What it is**: ShadowMem is a security camera that inspects your computer's RAM (volatile working memory) while programs are actively running.
2. **The problem it solves**: Traditional antiviruses scan files sitting on your hard drive; modern "fileless" malware runs strictly inside memory without ever saving a **`.exe`** to disk.
3. **How it inspects**: It uses standard Windows user-mode operating system tools (**`VirtualQueryEx`**) to check every memory room allocated to running programs.
4. **The telltale red flag**: Legitimate programs keep code locked down; malicious code injects into memory pages marked **RWX** (Read + Write + Execute) so it can rewrite its own code on the fly.
5. **The smart filter (JIT)**: Web browsers (Chrome, Edge) and runtimes (Node.js, Java) also use executable memory to run fast scripts ("Just-In-Time" compilation); ShadowMem recognizes them and ignores them so you don't get false alarms.
6. **The decision engine**: It feeds 7 specific memory measurements into a trained machine-learning model (**`HistGradientBoosting`**) to decide if a program is benign, suspicious, or malicious.
7. **The response**: If a threat is detected, it freezes all threads of that program in place (**`SuspendThread`**) so the attack cannot continue, without crashing Windows.
8. **What it does NOT have**: It has **zero kernel drivers** and **zero Ring-0 code**. It runs entirely in user space with administrator rights.
9. **How it talks**: A FastAPI Python backend inspects the operating system, exposes **`/api/v1`** REST routes and a WebSocket feed, and serves a Next.js web dashboard.
10. **The demo takeaway**: You get real-time memory forensics, visual radar mapping, and one-click process containment in a lightweight dashboard.

1. Global Parts on Every Page

These elements surround every screen inside the navigation frame.

1.1 Sidebar Navigation

• **Element Name**: **`Sidebar Navigation`** (Overview, Live Map, Process Table, Analytics, Activity Log)

• **Tag**: **`REAL`**

• **Plain Words**: The main menu on the left side of the screen.

• **What Happens When Clicked**: Navigates between the 5 dashboard routes (**`/`**, **`/live-map`**, **`/process-table`**, **`/analytics`**, **`/kernel-log`**).

• **Source & API**: **`frontend/src/components/Sidebar.tsx#L40-L75`**. Routes are handled client-side via Next.js **`Link`**.

• **Formula / Logic**: Highlights the active item by comparing **`pathname === item.href`**.

• **Before Scan vs After Scan**: Stays accessible and identical before and after scans.

1.2 Sidebar Engine Box

• **Element Name**: **`Engine`** box (Mode, Model, Last scan)

• **Tag**: **`REAL`**

- **Plain Words**: A small status box at the bottom of the sidebar that proves ShadowMem is running honestly.
- **What Happens When Clicked**: Not clickable; displays static status and scan timestamps.
- **Source & API**: `frontend/src/components/Sidebar.tsx#L115-L134`.
- **Formula / Logic**:
- `Mode`: Static string `"user-mode (VirtualQueryEx)"`.
- `Model`: Static string `"HistGradientBoosting"`.
- `Last scan`: Listens to `shadowmem:scan` window event dispatched by `fetchScan()` in `frontend/src/lib/api.ts#L92`. Formats `Date.now()` into `HH:MM:SS`.
- **Before Scan vs After Scan**:
- *Before Scan*: Shows `Last scan: none`.
- *After Scan*: Shows `Last scan: 11:53:39 AM` (real time of last completed scan).

1.3 Top Bar Greeting & Clock

- **Element Name**: `Operational Greeting` ("Good afternoon, Commander", etc.) & UTC Clock
- **Tag**: `DERIVED`
- **Plain Words**: Shows a friendly greeting based on local time and displays a live digital UTC clock.
- **What Happens When Clicked**: Not clickable.
- **Source & API**: `frontend/src/components/Header.tsx#L25-L45`.
- **Formula / Logic**: Checks `new Date().getHours()` (<12 = Morning, <18 = Afternoon, else Evening). Runs a 1-second `setInterval` formatting UTC time.
- **Before Scan vs After Scan**: Ticks continuously regardless of scan status.

1.4 Top Bar Search Input

- **Element Name**: `Search process name, PID...`
- **Tag**: `NOT WIRED`
- **Plain Words**: A placeholder global search bar in the header.
- **What Happens When Clicked**: Nothing. The HTML `<input>` tag contains the `disabled` property.
- **Source & API**: `frontend/src/components/Header.tsx#L129-L135`.
- **Formula / Logic**: Static disabled HTML field. (Note: Search on individual pages like Process Table and Activity Log is fully functional).
- **Before Scan vs After Scan**: Always disabled.

1.5 Scan Mode Pill

- **Element Name**: `ON-DEMAND SCAN` / `SCANNER OFFLINE`
- **Tag**: `DERIVED`
- **Plain Words**: Tells you whether the frontend can talk to the Python backend.
- **What Happens When Clicked**: Not clickable.
- **Source & API**: `frontend/src/components/Header.tsx#L141-L152`. Driven by `isConnected` from `ConnectionContext.tsx`.
- **Formula / Logic**: If backend API is reachable (`isConnected === true`), shows orange/lime dot with `"ON-DEMAND SCAN"`. If connection drops, turns red and reads `"SCANNER OFFLINE"`.
- **Before Scan vs After Scan**: Shows `"ON-DEMAND SCAN"` as long as backend server is alive.

1.6 Scan Status Badge

- **Element Name**: `Status: Nominal` / `Status: Scanning memory...` / `Status: Disconnected`
- **Tag**: `DERIVED`

- **Plain Words**: Shows what the forensic engine is doing right this second.
- **What Happens When Clicked**: Not clickable.
- **Source & API**: `frontend/src/components/Header.tsx#L155-L165`.
- **Formula / Logic**:
- If `!isConnected` -> `"Disconnected"` (red text).
- If `isScanning` -> `"Scanning memory..."` (orange animated text).
- Else -> `"Nominal"` (lime green text).

(Judge note: This header status only reflects connection and scan execution; it does not change to "Critical" when threats are found—threat status is displayed on the cards and Threat Posture panel).

- **Before Scan vs After Scan**: Shows `"Nominal"` before scan, `"Scanning memory..."` while scan runs, and returns to `"Nominal"` after scan finishes.

1.7 Density Toggle

- **Element Name**: `Detailed` / `Compact`
- **Tag**: `REAL`
- **Plain Words**: A view switch like changing email view from "comfortable" to "compact".
- **What Happens When Clicked**: Switches card padding, chart heights, and table row heights between full size and condensed size.
- **Source & API**: `frontend/src/components/Header.tsx#L168-L193` via `DensityContext.tsx`.
- **Formula / Logic**: Sets `density = 'detailed' | 'compact'`. Applies Tailwind classes (e.g. `p-4 h-32` vs `p-2.5 h-24`).
- **Before Scan vs After Scan**: Works instantly at any time.

1.8 Notification Bell

- **Element Name**: `Security Notifications` Bell
- **Tag**: `REAL`
- **Plain Words**: A dropdown tray showing the 5 most recent security alert events.
- **What Happens When Clicked**: Opens a floating popover showing recent alerts and unread count. Clicking "Mark all read" clears the red badge.
- **Source & API**: `frontend/src/components/Header.tsx#L196-L281` via `NotificationsContext.tsx`.
- **Formula / Logic**: `unreadCount` shows number of alerts added since dropdown was last opened. Shows `alert.title`, timestamp, and severity indicator.
- **Before Scan vs After Scan**: Empty before scan; fills when alerts are generated or WebSocket events arrive.

1.9 Header Export JSON Button

- **Element Name**: `Export JSON` (in top right header)
- **Tag**: `NOT WIRED`
- **Plain Words**: A placeholder button in the top navigation bar.
- **What Happens When Clicked**: Nothing. Has no `onClick` event handler attached.
- **Source & API**: `frontend/src/components/Header.tsx#L285-L291`.
- **Formula / Logic**: Pure JSX button markup without JavaScript logic. *(Tip: Use "Export CSV" on the Process Table page or "Export Scope JSON" on the Live Map page instead—those are fully wired).*
- **Before Scan vs After Scan**: Does not do anything in either state.

1.10 Page Subheader Chips

- **Element Name**: `LIVE API CONNECTED` & `WS LINK`
- **Tag**: `LIVE API CONNECTED` = `HARDCODED / DECORATIVE`; `WS LINK` = `REAL`

- **Plain Words**: Small tags indicating network connectivity to the backend.
- **What Happens When Clicked**: Not clickable.
- **Source & API**: `frontend/src/app/page.tsx#L320-L340`.
- **Formula / Logic**:
- `LIVE API CONNECTED`: Permanently rendered JSX badge with a pulsing green dot.
- `WS LINK`: Conditionally rendered `{wsConnected && <span ...>WS LINK</span>}`. Appears only when the `/ws/alerts` WebSocket connection is open.
- **Before Scan vs After Scan**: Visible as long as the page is open.

2. Overview Page (/)

The primary executive dashboard displaying top-level metrics, the spatial process grid, posture gauge, analytics, and telemetry feed.

2.1 "Run Forensics Scan" Button

- **Element Name**: `Run Forensics Scan`
- **Tag**: `REAL`
- **Plain Words**: The main trigger button that starts the memory inspection across the computer.
- **What Happens When Clicked**: Sends an HTTP `GET /api/v1/scan` request to the backend. Turns the button into a spinning spinner reading `"Auditing Volatile VADs..."` and updates every card and graph when the scan completes (~3 to 6 seconds).
- **Source & API**: `frontend/src/app/page.tsx#L134-L220`.
- **Formula / Logic**: Measures execution speed via `performance.now()`. Receives JSON containing `summary` and `processes[]`, and populates local React state.
- **Before Scan vs After Scan**: Active button before scan; during scan it is disabled; after scan it returns to active so you can scan again.

2.2 "Reset Baseline" Button

- **Element Name**: `Reset Baseline`
- **Tag**: `REAL`
- **Plain Words**: Clears current scan results from your screen and resets the dashboard to its fresh, un-scanned view.
- **What Happens When Clicked**: Resets React state: `hasScanned = false`, `summary = null`, `processes = []`.
- **Source & API**: `frontend/src/app/page.tsx#L339-L351`.
- **Formula / Logic**: Only appears when `hasScanned === true`.
- **Before Scan vs After Scan**: Hidden before first scan; visible after scan runs.

2.3 The Four Top Stat Cards

Located in `frontend/src/components/dashboard/StatCards.tsx`.

Card 1: Scanned Processes

- **Tag**: `REAL`
- **Plain Words**: How many running programs ShadowMem inspected during the scan.
- **Formula / Logic**: `summary.total_scanned`. Subtext shows `summary.clean_count` verified clean.
- **Before Scan**: Shows empty card with `"Engine waiting to benchmark"`.
- **After Scan**: e.g., `131 processes`, subtext: `100 verified clean`.

Card 2: Active Threats

- **Tag**: `REAL`
- **Plain Words**: How many programs have confirmed malicious code injected into them.

- **Formula / Logic**: `summary.malicious_count`. If > 0, turns red and shows `"Threat containment required"`. If 0, turns green and shows `"Zero active threats"`.
- **Before Scan**: Shows `"No critical anomalies registered"`.
- **After Scan**: e.g., `1 malicious PIDs`, subtext: `"User-mode suspend recommended"`.

Card 3: Injections Flagged

- **Tag**: `DERIVED`
- **Plain Words**: The combined number of suspicious memory anomalies and confirmed threats detected.
- **Formula / Logic**: `summary.anomalies_detected` (`malicious_count + suspicious_count`). Subtext shows breakdown: `${suspicious_count} suspicious / ${jit_engine_count} JIT ignored`.
- **Before Scan**: Shows `"Memory anomaly monitoring ready"`.
- **After Scan**: e.g., `2 anomalies`.

Card 4: Last Scan Duration

- **Tag**: `REAL`
- **Plain Words**: How long the entire scan took, and the average inspection speed per program.
- **Formula / Logic**: Total elapsed milliseconds (`elapsed = performance.now() - t0`). Underneath, per-process average is calculated: `(elapsed / total_scanned).toFixed(1)` ms.
- **Before Scan**: Shows `"Engine waiting to benchmark"`.
- **After Scan**: e.g., `5241 ms`, subtext: `~40.0ms avg per process`.

2.4 Live Process Memory Map Tiles & Filter Tabs

- **Element Name**: `Live Process Memory Map`
- **Tag**: `REAL`
- **Plain Words**: An interactive grid where each box represents a running program on your PC.
- **What Happens When Clicked**:
- Clicking filter tabs (`ALL`, `THREATS`, `JIT`, `CLEAN`) filters which process boxes are visible and shows tab counts.
- Clicking any process box opens its **Process Forensic Dossier modal**.
- Hovering a box reveals its details in the bottom inspector bar.
- **Source & API**: `frontend/src/components/dashboard/LiveProcessMapPanel.tsx#L85-L195`.
- **Formula / Logic**: Each tile displays `PID`, process `name`, memory allocated (`allocated_kb`), threat confidence (`threat_score.toFixed(1)%`), and a color dot:
- Red = Malicious (`status === "Malicious (Injected)"`)
- Orange = Suspicious (`status === "Suspicious"`)
- Blue = JIT Engine (`status === "Legitimate JIT Engine"`)
- Green = Clean (`status === "Clean"`)
- **Before Scan vs After Scan**: Before scan: shows dashed placeholder `"Process Radar Standby"`. After scan: renders dozens of interactive process tiles.

2.5 Threat Posture Overview Panel

- **Element Name**: `Threat Posture Overview`
- **Tag**: `DERIVED`
- **Plain Words**: A circular progress ring showing what percentage of your computer is healthy and safe.
- **What Happens When Clicked**: Not clickable.
- **Source & API**: `frontend/src/components/dashboard/ThreatPosturePanel.tsx#L48-L165`.

- **Formula / Logic**:
- `cleared = clean_count + jit_engine_count`
- `clearRate = ((cleared / total_scanned) * 100).toFixed(1)%`
- The SVG circle circumference is calculated via math: `strokeDashoffset = circumference - (clearRate / 100) * circumference`.
- Tile 1 (`Processes Cleared`): Shows `${cleared} / ${total_scanned}`.
- Tile 2 (`Threat Level`): Shows `"LOW"` (green) if `malicious_count === 0`, or `"ELEVATED"` (red) if `malicious_count > 0`.
- Footer bar shows: `Model: HistGradientBoosting (7 features)`.
- **Before Scan vs After Scan**: Before scan: shows `"Posture Assessment Pending"`. After scan: SVG circle animates and displays clear rate (e.g. `98.5%`).

2.6 Process Status Distribution (Donut Chart)

- **Element Name**: `Process Status Distribution`
- **Tag**: `DERIVED`
- **Plain Words**: A colored donut chart breaking down your processes into 4 categories.
- **What Happens When Clicked**: Hovering slices displays an interactive tooltip with category counts.
- **Source & API**: `frontend/src/components/dashboard/AnalyticsRow.tsx#L37-L165` using Recharts.
- **Formula / Logic**: Uses values directly from `summary`: Clean (Green `#3D8B60`), JIT Engines (Blue `#4A7BB0`), Suspicious (Orange `#E68A2E`), Malicious (Red `#D6453D`). Center of the donut shows `summary.total_scanned` Audited.
- **Before Scan vs After Scan**: Placeholder before scan; renders donut after scan.

2.7 Threats Over Time (Area Chart)

- **Element Name**: `Threats Over Time`
- **Tag**: `DERIVED`
- **Plain Words**: A timeline graph showing whether threats are increasing or decreasing with each scan you run.
- **What Happens When Clicked**: Hovering points shows scan duration and threat counts.
- **Source & API**: `frontend/src/components/dashboard/AnalyticsRow.tsx#L169-L250`.
- **Formula / Logic**: Stored in React state `scanHistory[]`. Every time you click "Run Forensics Scan", a new record is appended: `{ scanIndex, timeLabel, threats, malicious, totalScanned }`.
- **Before Scan vs After Scan**: Shows `"Velocity Trend Awaiting Cycles"` before scan; plots curve after scans.

2.8 Top Threat Categories

- **Element Name**: `Top Threat Categories`
- **Tag**: `DERIVED`
- **Plain Words**: A progress bar ranking the specific attack techniques discovered during the scan.
- **What Happens When Clicked**: Not clickable.
- **Source & API**: `frontend/src/components/dashboard/AnalyticsRow.tsx#L48-L73`.
- **Formula / Logic**: Iterates over all flagged processes, groups by `p.classification.name` and MITRE ID (e.g. `T1055.001`), and calculates percentage of total threats: `percentage = (count / totalFlagged) * 100`.
- **Before Scan vs After Scan**: Shows `"No Threat Injections Found"` if 0 threats exist; shows ranked list when threats are detected.

2.9 Recent Forensic Alerts Panel

- **Element Name**: `Recent Forensic Alerts`
- **Tag**: `REAL`
- **Plain Words**: A security feed listing every flagged process alert in chronological order.

- **What Happens When Clicked**:
- Clicking an alert associated with a PID automatically opens the Process Dossier for that process.
- Clicking the "Clear" button clears the alert list.
- **Source & API**: `frontend/src/components/dashboard/RecentAlertsPanel.tsx#L85-L135`.
- **Formula / Logic**: Fed by React `alerts[]` state. If scan finds threats, adds alerts. If scan finds 0 threats, adds a verified event: `"Scan completed: 131 processes inspected. Zero threats detected."`
- **Before Scan vs After Scan**: Shows `"No Active Threat Alerts"` before scan; populates with real events after scan.

2.10 Scan Activity Feed

- **Element Name**: `Scan Activity Feed`
- **Tag**: `REAL`
- **Plain Words**: A live scrolling terminal showing scan commands, isolation events, and WebSocket messages as they happen.
- **What Happens When Clicked**:
- `PAUSE / RESUME`: Toggles auto-scroll/freeze of incoming logs.
- `AUTO-SCROLL`: Enables/disables snapping to the bottom.
- `CLEAR`: Empties the current log view buffer.
- **Source & API**: `frontend/src/components/dashboard/KernelTelemetryFeed.tsx#L43-L148`.
- **Formula / Logic**: React state `kernelLogs[]`. Appended whenever `handleExecuteScan()` starts, finishes, or catches an error, or when an isolation action executes. Retains up to 120 lines in memory.
- **Before Scan vs After Scan**: Displays `"Log stream buffer empty. Awaiting scan activity..."` on clean start; fills with real scan logs upon execution.

3. Live Map Page (`/live-map`)

A circular visual radar projecting all processes in memory onto concentric threat circles.

3.1 The Polar Radar Canvas

- **Element Name**: Polar Radar Scope
- **Tag**: `DERIVED`
- **Plain Words**: Like an air-traffic controller's radar screen, but for memory space.
- **What Happens When Clicked**:
- Clicking any blip/dot opens the **Process Forensic Dossier modal** for that process.
- Moving mouse over any blip displays a hovering quick-inspect card.
- **Source & API**: `frontend/src/app/live-map/page.tsx#L160-L330`.
- **Formula / Logic**:
- Uses HTML5 Canvas (`<canvas>`).
- Radius `r` is determined by threat severity.
- Angle `theta` is calculated deterministically from a hash of the process name and PID (`hashString(p.name + p.pid) % 360`) so each process stays in a fixed position on the radar screen.

3.2 What the 4 Defense Rings Mean

1. **TIER 0 (Innermost circle, `r = 0.28`)**:
- **Meaning**: Confirmed in-memory threats (`status === "Malicious (Injected)"`). Placed closest to the bullseye because they represent active critical danger.

- **Color**: Glowing Red (**`#D6453D`**).

2. **TIER 1 (Second circle, `r = 0.48`)**:

- **Meaning**: Suspicious processes (**`status === "Suspicious"`**). Memory protection was flipped (e.g., Read/Write changed to Execute), but malicious classification is not 100% confirmed.
- **Color**: Amber (**`#E68A2E`**).

3. **TIER 2 (Third circle, `r = 0.70`)**:

- **Meaning**: Legitimate JIT engines (**`jit_engine === true`**, e.g. Node.js, Chrome V8, Java).
- **Color**: Blue (**`#4A7BB0`**). They have executable memory, but are benign.

4. **TIER 3 (Outermost circle, `r = 0.92`)**:

- **Meaning**: Completely benign user-mode programs (**`status === "Clean"`**).
- **Color**: Emerald green (**`#3D8B60`**).

3.3 Center Origin ("SYSTEM ORIGIN")

- **Element Name**: **`SYSTEM ORIGIN`** Anchor
- **Tag**: **`DECORATIVE`**
- **Plain Words**: The orange center dot at coordinate **`(0, 0)`** of the radar screen representing the operating system root.
- **What Happens When Clicked**: Not clickable.
- **Source & API**: **`frontend/src/app/live-map/page.tsx#L251-L265`**.
- **Formula / Logic**: Canvas arc filled at center coordinates **`(cx, cy)`**.

3.4 Rotating Radar Sweep

- **Element Name**: Rotating Radar Beam
- **Tag**: **`DECORATIVE`**
- **Plain Words**: The glowing green sweeping line rotating around the circle like a real submarine sonar or airport radar.
- **What Happens When Clicked**: Can be enabled or disabled by clicking the **Radar Sweep** button in the header.
- **Source & API**: **`frontend/src/app/live-map/page.tsx#L235-L250`**.
- **Formula / Logic**: Computed using **`requestAnimationFrame`**: **`sweepAngle = (Date.now() / 3500) (Math.PI 2)`**. It draws a linear gradient line with fading trail. It is purely visual and does not dictate scan timing.

3.5 Action Controls

- **Radar Sweep Toggle**: Toggles rotation animation on/off (**`REAL`**).
- **ALL / THREAT / JIT / CLEAN Filter Buttons**: Filters which dots are drawn on canvas (**`REAL`**).
- **Run Forensics Scan**: Calls **`fetchScan()`** and re-projects all canvas nodes (**`REAL`**).
- **Export Scope JSON**: Generates and downloads a **`.json`** file containing all radar coordinates and process records client-side (**`REAL`**, **`frontend/src/app/live-map/page.tsx#L140-L155`**).

4. Process Table Page (`/process-table`)

A deep, filterable spreadsheet-style view designed for SOC analysts who want to inspect raw numbers.

4.1 Table Columns Explained

Column Name	Meaning in Plain English	Source in API	Tag
PID	Process ID: the unique numeric badge Windows assigns to every running program.	**`process.pid`**	**REAL**
Process Name	The file name of the program (e.g. **`chrome.exe`**, **`svchost.exe`**).	**`process.name`**	**REAL**
Executable Path	The exact folder on disk where this program lives. Blank if unbacked.	**`process.exe`**	**REAL**
RWX Regions	Memory rooms marked Read + Write + Execute at the same time. The #1 red flag.	**`process.rwx_count`**	**REAL**
Unbacked Exec	Code running in memory that does NOT belong to any file on your hard drive.	**`process.unbacked_exec_count`**	**REAL**
Total RWX Bytes	Total size in bytes of all RWX memory rooms added together.	**`process.total_rwx_bytes`**	**REAL**
Allocated KB	Total memory allocated to volatile executable pages in Kilobytes.	**`process.allocated_kb`**	**DERIVED**
Threads	How many independent workers/tasks are running inside this program.	**`process.num_threads`**	**REAL**
Unbacked Starts	Threads whose starting instruction points into unbacked memory (shellcode clue).	**`process.thread_unbacked_start`**	**REAL**
VAD Flips	Memory regions whose permissions were changed while running (e.g. RW -> RX).	**`process.vad_protection_anomaly`**	**REAL**
Threat Score	Confidence percentage (0-100%) calculated by the ML model.	**`process.threat_score`**	**DERIVED**
Status	Final verdict: Malicious, Suspicious, JIT Engine, or Clean.	**`process.status`**	**DERIVED**
Actions	Direct button to Isolate or Resume the target process.	Calls **`/api/isolate`** or **`/resume`**	**REAL**

4.2 Interactive Table Controls

- **Column Sorting**: Clicking any column header (e.g. PID, RWX, Threat Score) sorts the entire table ascending or descending (**`sortField`**, **`sortAsc`**).
- **Search Input**: Filters table in real time as you type by program name, PID number, or disk path.
- **Filter Tabs**: Tabs for **`All`**, **`Malicious`**, **`Suspicious`**, **`JIT`**, and **`Clean`** with live counts.
- **Row Click**: Clicking anywhere on a process row opens its full **Process Forensic Dossier modal**.
- **Export CSV Button**: Exports all visible table data directly to a downloadable **`.csv`** file (**`shadowmem_forensics_<timestamp>.csv`**).

5. Analytics Page (`/analytics`)

The executive reporting view summarizing detection metrics, models, and technique breakdown.

5.1 KPI Metric Cards

Located at **`frontend/src/app/analytics/page.tsx#L195-L230`**.

1. **Overall Clear Rate**:
   - **Tag**: **`DERIVED`**
   - **Plain Words**: Percentage of processes with zero threats.
   - **Formula**: **`((clean_count + jit_engine_count) / total_scanned) * 100`**%.
   - **Before Scan vs After Scan**: Shows **`100%`** / **`"Awaiting scan audit"`** before scan; shows exact computed percentage after scan (e.g. **`98.5%`**).
2. **System Posture**:
   - **Tag**: **`REAL`**

- **Plain Words**: Executive status verdict.
- **Formula**: Shows **`"PROTECTED"`** (green) if **`summary.malicious_count === 0`**, or **`"THREAT DETECTED"`** (red) if **`summary.malicious_count > 0`**.
- **Before Scan vs After Scan**: Shows **`"STANDBY"`** before scan; updates after scan.

(Honesty Note: Previous hardcoded metric cards claiming "ML Detection Accuracy 99.4%" and "False Positive Estimate < 0.2%" were audited and deleted during Step B because they were static JSX strings rather than dynamic live computations).

5.2 Status Distribution & Threat Trend Charts

- **Status Distribution**: Recharts interactive Donut chart dividing processes into Clean, JIT, Suspicious, and Malicious (**`DERIVED`**).
- **Threat Trend**: Recharts Area chart plotting how many threats were found across your scan history (**`DERIVED`**).
- **Memory Metric Breakdown**: Recharts Bar chart comparing RWX count, unbacked regions, thread anomalies, and VAD flips side-by-side (**`DERIVED`**).

5.3 MITRE Classification Breakdown Table

- Lists attack techniques mapped to the official MITRE ATT&CK framework (e.g., **`T1055.001`** Dynamic-link Library Injection, **`T1620`** Reflective Code Loading).
- Displays the number of flagged processes matching each pattern and the corresponding mitigation advisory (**`REAL`**).

6. Alerts Page (`/alerts`)

The live incident queue showing incoming security warnings and synthetic simulation triggers.

6.1 Severity Filter Pills

- **Pills**: **`ALL`**, **`CRITICAL`**, **`WARNING`**, **`NOMINAL`**, **`INFO`**.
- **Tag**: **`REAL`**
- **Plain Words**: Filters the alert cards by threat level.
- **Formula / Logic**: Compares **`alert.severity === activeFilter`**. Displays pill count badge based on **`alerts.filter(...)`**.

6.2 "Trigger Simulated Threat" Button

- **Element Name**: **`Simulate Threat Alert`**
- **Tag**: **`REAL (Simulated Trigger)`**
- **Plain Words**: A demo trigger button built specifically for hackathons and sales demonstrations.
- **What Happens When Clicked**: Sends an HTTP **`POST /api/v1/simulate/alert`** with JSON **`{ "scenario": "cobalt_strike_reflective_injection", "severity": "critical" }`**.
- **Why It Exists & What It Really Does**:
- Hackathon judges cannot safely allow you to detonate live Cobalt Strike ransomware or real malware payloads on the evaluation laptop.
- This button triggers the backend's **`simulate_alert()`** function in **`services/alerts.py`**, which synthesizes a realistic forensic detection alert, broadcasts it across the live WebSocket **`/ws/alerts`**, and adds it to the audit stream.
- It proves to the judge that the WebSocket broadcast pipeline and real-time frontend alert banner work end-to-end without needing to execute dangerous shellcode.

6.3 Clear Alerts Button

- **Element Name**: **`Clear Alerts`**
- **Tag**: **`REAL`**
- **Plain Words**: Empties the current alert stream queue from screen.

7. Activity Log Page (`/kernel-log`)

A deep forensic terminal providing an audit log of scanning and containment actions.

7.1 Page Header & Engine Badge

- **Page Title**: `Live Activity Log & Audit Stream`
- **Badge**: `USER-MODE ENGINE` (Tag: `HARDCODED / DECORATIVE`). Proves user-mode operation.
- **Subtitle**: *"Direct output stream of memory page validations, thread walkthroughs, and forensic scan events."*

7.2 Terminal Output Body

- **Tag**: `REAL`
- **Plain Words**: The dark output window where log lines appear.
- **Content**: Shows timestamps, tags (`[SCAN]`, `[SYS]`, `[ISOLATE]`, `[ML]`), and log messages.
- **Before Scan vs After Scan**: Clean empty state (`"Terminal buffer empty..."`) on start. When a scan starts, logs the scan initiation; when completed, logs process count and elapsed milliseconds. When a process is suspended/resumed, logs the exact PID and thread action.

7.3 Action Buttons

- **PAUSE / RESUME**: Pauses incoming log stream rendering (`REAL`).
- **AUTO-SCROLL**: Snaps window to bottom as new events arrive (`REAL`).
- **CLEAR**: Clears terminal lines (`REAL`).
- **Export Log**: Generates and downloads a `.log` text file (`shadowmem_activity_<timestamp>.log`) containing the full chronological terminal contents (`REAL`).

8. Process Detail Window (Dossier Modal)

Opened by clicking any process row, tile, or radar blip. Located in `frontend/src/components/dashboard/ProcessDossierModal.tsx`.

8.1 Process Identity & Executable Path

- **Process Name & PID**: Displays name (e.g. `malware_target.exe`) and PID (`REAL`).
- **Disk Path**: Shows the binary's path on the hard drive (e.g. `C:\Windows\System32\...`). If a process has no disk path, displays `"No disk image path (in-memory unbacked module)"` (`REAL`).
- **Status Badge & Threat Score**: Shows verdict (`Malicious (Injected)`, `Suspicious`, etc.) and score (e.g. `87.5%`) (`DERIVED`).

8.2 The 7-Feature Volatile Memory Forensic Vector

The 7 exact inputs fed into the machine learning classifier:

1. **RWX Page Count**: Number of memory pages marked Read, Write, and Execute. Red alert if > 2.
2. **Unbacked Exec Regions**: Memory pages marked executable that do not map to any `.exe` or `.dll` on disk. Red alert if > 0.
3. **RWX Allocated KB**: Total memory size of RWX regions in Kilobytes and bytes. Red alert if > 256 KB.
4. **Thread Count**: Number of active threads running in this process.
5. **Unbacked Thread Starts**: Threads whose instruction pointer (`EIP`/`RIP`) started inside private memory rather than a recognized DLL export. Red alert if > 0.
6. **VAD Protection Flips**: Memory regions whose access flags were dynamically changed during execution (e.g. `PAGE_READWRITE` altered to `PAGE_EXECUTE_READWRITE`). Red alert if > 0.
7. **Known JIT Parent**: Whether the process is a recognized child or runtime of an engine like V8 or JVM (`jit_engine === true`).

8.3 MITRE ATT&CK Classification Dossier

- Shows the matched attack technique name, MITRE ID (e.g. **`T1055.001`**), attack mechanism explanation, and a specific remediation instruction (e.g. *"Perform user-mode thread suspension immediately"*).

8.4 The "Isolate Target Process" & "Resume Process" Buttons

- **Tag**: **`REAL`**
- **Plain Words**: The emergency kill-switch / freeze button.
- **What Windows Calls Are Executed**:

1. Calls **`OpenProcess(PROCESS_SUSPEND_RESUME, False, pid)`** in **`services/containment.py#L143`**.
2. Enumerates every active thread in the process via **`psutil.Process(pid).threads()`**.
3. For each thread, calls **`OpenThread(THREAD_SUSPEND_RESUME, False, tid)`** and **`SuspendThread(thread_handle)`**.
4. To undo, calls **`ResumeThread(thread_handle)`** on each thread.

- **What It Looks Like on the Computer**:
- The process does **not** crash or terminate; it is **frozen in time**.
- Its CPU consumption drops to exactly **0.0%**.
- Its GUI windows stop responding and will not process user clicks.
- The process remains in memory so forensic analysts can dump its RAM for analysis without the malware causing further harm.
- **Safety Guards**:
- ShadowMem's own PID and parent PID cannot be isolated (self-protection).
- Protected Windows system processes (**`csrss.exe`**, **`smss.exe`**, **`lsass.exe`**, **`services.exe`**, **`wininit.exe`**) are blocked from isolation so the OS cannot be crashed.
- **How to Undo**: Clicking the button again (now labeled **`"Resume Process (Un-isolate)"`**) calls **`ResumeThread()`** on each thread handle, instantly restoring normal execution.

Section A: "What Happens When I Click Run Forensics Scan"

The complete end-to-end journey in 8 simple numbered steps:

`

[User Clicks "Run Forensics Scan"]

■

▼

1. Next.js triggers **`handleExecuteScan()`** in page.tsx

■

▼

2. HTTP GET /api/v1/scan sent to FastAPI backend

■

▼

3. Backend calls **`enumerate_processes()`** via psutil (collects ~130 PIDs)

■

▼

4. Backend opens each process via **`OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ)`**

■

▼

5. Memory walkthrough: calls Win32 **`VirtualQueryEx`** to inspect every memory region

-> Counts RWX pages, checks unbacked MEM_PRIVATE, inspects thread start addresses

■

▼

6. 7 features assembled -> passed into HistGradientBoostingClassifier model

-> Predicts class: Benign (Clean/JIT) vs Malicious/Suspicious

-> Calculates threat score (0% to 100%)

■

▼

7. Backend compiles JSON response (**`summary`** + **`processes[]`**) and returns HTTP 200

■

▼

8. Frontend receives JSON:

-> Updates StatCards, Live Map tiles, Radar Canvas, Donut, and Posture Ring

-> If anomalies found, adds alert; if clean, logs verified clean audit event

`

Section B: Worked Example (e.g. 131 Scanned Processes)

1. What Exactly Is Counted?

When the card says **131 Scanned Processes**, it means ShadowMem iterated through the Windows process list and successfully inspected the memory topology of 131 active user-mode processes.

2. Why Does the Count Fluctuate by 1 or 2 Between Scans?

Windows is constantly launching and terminating short-lived background tasks:

- Windows Update workers (**`TiWorker.exe`**, **`TrustedInstaller.exe`**).
- Telemetry tasks, background Chrome audio or rendering sub-processes.
- Antivirus background health checks.

Because these background tasks spawn and exit naturally, running two scans 10 seconds apart might return 131 on the first scan and 133 on the second scan. This is normal, healthy operating system behavior.

3. What Does "Denied / Skipped" Mean?

Certain processes cannot be inspected:

- **System Idle Process (PID 0)** and **System (PID 4)**: These are not real user-mode programs; they are kernel constructs.
- **Protected Processes (Antivirus, Windows Defender)**: Run under Windows Protected Process Light (PPL). Calling **`OpenProcess`** on them returns **`Win32 Error 5: ERROR_ACCESS_DENIED`**.

ShadowMem safely skips these without crashing and logs them as skipped.

4. What Does the JIT Number Mean & Why Are They Not Threats?

In modern computing, **JIT** stands for **Just-In-Time Compilation**:

- **Real-Life Analogy**: A regular program is like a printed book in English. A script (like JavaScript on a web page) is written in a foreign language. A JIT engine is a high-speed translator that reads the foreign language and immediately writes English notes in the margins so it can read it super fast.

- To do this, web browsers (Chrome, Edge, Firefox), Node.js, and Java allocate memory pages where they can **write** translated machine code and then **execute** it.
- To a naive scanner, this looks identical to malware injecting shellcode!
- **ShadowMem's Solution**: It inspects process parentage, executable path, known runtime signatures, and thread properties. If the process is a known JIT engine (e.g. Chrome's rendering engine), it classifies it as **`Legitimate JIT Engine`**. It is excluded from the threat count so you don't receive 30 false positive alerts every time you open a web browser.

Section C: Plain-Language Glossary

1. **Process**: A running software program on your computer (e.g., Spotify, Chrome).

Analogy: An office room where work is being done.

2. **PID (Process Identifier)**: The badge number Windows pins on every running program.

Analogy: An employee ID badge.

3. **Thread**: An individual worker inside a program doing a specific task.

Analogy: A clerk sitting at a desk inside that office room.

4. **Virtual Memory**: The scratchpad address space Windows gives to each program so it doesn't mess with other programs' data.

Analogy: A private whiteboard assigned to each office.

5. **Memory Region**: A contiguous block of pages inside that virtual scratchpad.

Analogy: A section or paragraph on that whiteboard.

6. **Executable Memory**: Memory marked with permission to run code instructions, not just hold text or images.

Analogy: A recipe card on the whiteboard that the chef is allowed to cook from.

7. **RWX (Read-Write-Execute)**: A memory region that can be read, written to, and executed at the same time.

Analogy: An open bank vault where anyone can alter the ledger and immediately print money. In secure software engineering, memory should be Writable OR Executable, never both (**`W^X`**).

8. **Unbacked Memory**: Executable memory that is floating in RAM without any corresponding **`.exe`** or **`.dll`** file saved on your hard drive.

Analogy: A ghost employee sitting in the office who isn't on the company payroll.

9. **JIT (Just-In-Time Compiler)**: A speed booster used by browsers (JavaScript V8) and Java to compile code on the fly.

Analogy: A simultaneous court interpreter translating words instantly into a microphone.

10. **Shellcode**: A tiny payload of raw machine code injected into memory to launch an attack.

Analogy: A sticky note slipped onto the chef's desk containing instructions to poison the soup.

11. **Fileless Malware**: Malware that exists only in RAM and never writes a malicious file to disk, bypassing traditional file-based antiviruses.

Analogy: A burglar who never leaves footprints or fingerprints because they only walk on air.

12. **Process Injection**: The act of one program surreptitiously injecting its code into another legitimate program's memory space.

Analogy: A criminal slipping counterfeit bills into a legitimate cashier's cash register.

13. **Containment (Isolation)**: Pausing all execution threads of a compromised process without shutting down the entire PC.

Analogy: Putting a suspect into handcuffs and hitting the pause button on their desk so they can't touch anything.

14. **Authenticode**: Microsoft's digital signature stamp proving an executable came from a verified publisher (like Microsoft or Google).

Analogy: A wax seal on an official government letter.

15. **SHA-256**: A cryptographic fingerprint of a file. If even one letter changes, the fingerprint changes completely.

Analogy: A human DNA fingerprint.

16. **MITRE ATT&CK**: The globally recognized encyclopedic taxonomy of cyber adversary behaviors and techniques.

Analogy: The FBI's standard classification handbook of criminal tactics.

17. **T1055.001 / T1620**:

- **`T1055.001`** (Dynamic-Link Library Injection): Forcing another process to load a malicious DLL.
- **`T1620`** (Reflective Code Loading): Loading and executing code directly from memory without registering it with the Windows operating system loader.

Section D: The 15 Hardest Questions a Judge Could Ask

Q1: "Is this tool operating at the kernel level or user level?"

- **Answer**: *"ShadowMem operates strictly in user mode. It has no kernel driver and no Ring-0 hooks. It relies on documented Win32 APIs like* **`VirtualQueryEx`** *and* **`OpenProcess`***. This makes it safer and easier to deploy without risking Blue Screen of Death (BSOD) crashes."*

Q2: "Where does the 99.4% detection accuracy claim come from?"

- **Answer**: *"That 99.4% figure was from our initial offline training set benchmark. Because it was an offline metric and not a dynamic real-time measurement of the live scan, we audited our dashboard and removed that tile to ensure 100% operational honesty. Our dashboard now shows live measurements: processes scanned, cleared, and active threats."*

Q3: "What machine learning model are you actually running?"

- **Answer**: *"We run a* **`HistGradientBoostingClassifier`** *wrapped in a scikit-learn* **`RobustScaler`** *pipeline, loaded from* **`shadowmem_model.pkl`***. It evaluates 7 volatile memory features per process and classifies them across 9 subclasses (5 benign, 4 malicious)."*

Q4: "How do you avoid flagging Chrome and Edge as malware when they allocate executable memory?"

- **Answer**: *"We inspect JIT runtime characteristics: known JIT parent process hierarchies, memory allocation sizes, and whether the executable is signed. If it matches legitimate browser runtimes (like Chrome's V8 engine), it is tagged as* **`Legitimate JIT Engine`** *and excluded from the threat tally."*

Q5: "What happens when you isolate a process? Does it kill it?"

- **Answer**: *"No, it does not terminate the process. It calls* **`SuspendThread`** *on every active thread inside the target PID. This freezes its execution at 0% CPU while leaving its memory intact for forensic memory dumping. We can instantly unfreeze it using* **`ResumeThread`***."*

Q6: "Can your isolation tool crash Windows if I accidentally isolate a system process?"

- **Answer**: *"No. In* **`services/containment.py`***, we implement hardcoded safeguard checks. We refuse to isolate ShadowMem itself, and we maintain an explicit protection blocklist against core Windows processes like* **`csrss.exe`**, **`smss.exe`**, **`lsass.exe`**, *and* **`services.exe`***."*

Q7: "How does the rotating radar sweep on the Live Map relate to the real scan?"

- **Answer**: *"The radar canvas is an interactive 2D spatial visualization. The position of each dot is real (determined by severity tier and a deterministic hash of the PID and name). However, the rotating sweep beam itself is a smooth HTML5 Canvas animation (*`requestAnimationFrame`*) designed for visual orientation, not a real-time radar hardware sensor."*

Q8: "Why does the top header have an Export JSON button that doesn't do anything?"

- **Answer**: *"The header Export JSON button is currently an un-wired UI button. However, the data export is fully functional in two other places: the Process Table page exports a complete CSV of all memory metrics, and the Live Map page exports a full scope JSON file."*

Q9: "What does the 'Simulate Threat Alert' button on the Alerts page actually do?"

- **Answer**: *"It calls* **`POST /api/v1/simulate/alert`** *with a synthetic Cobalt Strike reflective injection payload. It sends this over our live WebSocket to demonstrate how the SOC alert pipeline and incident response interface react in real time without having to execute dangerous malware live on your laptop."*

Q10: "If malware hooks `VirtualQueryEx`, how would ShadowMem detect it?"

- **Answer**: *"Because ShadowMem runs in user mode, an advanced rootkit or kernel-level malware that hooks system calls could hide memory pages. For a production enterprise deployment, ShadowMem would be paired with an EDR kernel sensor; ShadowMem's current architecture focuses on rapid, agentless user-mode volatile triage."*

Q11: "Why did my scanned process count change from 129 to 131 between two scans?"

- **Answer**: *"Windows is dynamic. Normal background worker processes—like Windows Update workers, Edge audio tabs, and telemetry tasks—spin up and exit continually. That small delta represents real OS activity."*

Q12: "Why are some processes showing 'No disk image path'?"

- **Answer**: *"In our scan, if a process was spawned from an unbacked memory injection or if its process handle could not resolve a file path due to permissions, it is flagged as unbacked. In-memory unbacked modules are a key indicator of Reflective DLL Injection (`T1620`)."*

Q13: "What happens if the backend server goes down while the frontend is open?"

- **Answer**: *"The frontend's `ConnectionContext` detects the failed heartbeat. The top status pill immediately switches to red (`SCANNER OFFLINE`), the status badge changes to `Disconnected`, and the error banner notifies the operator that the API link was lost."*

Q14: "How fast is a full memory scan?"

- **Answer**: *"On this machine, a complete scan of ~130 processes takes between 3.5 to 5.5 seconds total. That averages out to approximately 30 to 45 milliseconds per process."*

Q15: "What are the 7 features you feed into the ML model?"

- **Answer**: *"1: RWX page count, 2: unbacked executable regions, 3: total RWX bytes, 4: active thread count, 5: threads starting in unbacked memory, 6: known JIT parent flag, and 7: VAD protection anomalies."*

Section E: Weak Points & Honesty Register

Ranked by **embarrassment factor** if a judge questions it during a live demo:

Rank	Element	Current Tag	What It Does / Why It's Weak	How to Answer if Asked
#1	**Top Bar "Export JSON" Button** (`Header.tsx#L285`)	`NOT WIRED`	The button has no `onClick` handler. Clicking it does nothing.	*"That header button is a placeholder. For real exports, use the Export CSV button on the Process Table or Export Scope on the Live Map."*
#2	**Top Bar Global Search Bar** (`Header.tsx#L129`)	`NOT WIRED`	Has the HTML `disabled` attribute. You cannot click or type into it.	*"The global omni-search is currently disabled; table-level search on the Process Table and Activity Log is fully operational."*
#3	**"LIVE API CONNECTED" Chip** (`page.tsx#L320`)	`HARDCODED / DECORATIVE`	The chip is permanently rendered in JSX with a pulsing green dot, even if the backend is down.	*"That chip is a visual header label. The real connection state is shown by the Status pill right next to it and the dynamic WS LINK chip."*
#4	**Rotating Radar Beam** (`live-map/page.tsx#L235`)	`DECORATIVE`	The sweeping beam looks like a live hardware radar, but it's just a canvas animation.	*"The blips and defense rings represent real memory data, while the sweep is a visual radar animation to keep the scope readable."*
#5	**Header "Status: Nominal"** (`Header.tsx#L155`)	`DERIVED`	Shows "Nominal" even when threats are found (only changes to "Scanning memory..." or "Disconnected").	*"The header status badge tracks engine execution status (Scanning vs Idle). Threat severity is highlighted on the Threat Posture panel."*
#6	**"USER-MODE ENGINE" Tag** (`kernel-log/page.tsx#L164`)	`DECORATIVE`	A static green badge in the page header.	*"It is an architecture reminder confirming the engine runs in user space."*